CÓDIGOCLARO · RUTA PYTHON

# Tema 08 · Comprensiones y generadores

Cuaderno de estudio para DAM/DAW · 10 lecciones

## Qué vas a aprender

Comprensiones, yield, iteradores y evaluación perezosa.

Este cuaderno trabaja comprensiones y generadores en pasos cortos. La meta no es repetir palabras de memoria: es reconocer el problema, elegir una herramienta adecuada, explicar por qué funciona y comprobarla con datos distintos. Reserva una sesión para leer el modelo, otra para resolver el ejercicio y una tercera para intentar el laboratorio sin mirar la solución.

### Resultados de aprendizaje

- Explicar con tus palabras qué problema resuelve comprensiones y generadores y en qué parte de una aplicación DAM/DAW aparece.
- Leer el ejemplo, predecir el resultado y justificar cada decisión relevante.
- Resolver una práctica, comprobar casos normales y límite, y depurar un fallo con un método reproducible.

### Cómo estudiar este tema

Ten a mano un editor compatible con la tecnología y copia los ejemplos en un archivo nuevo. Antes de ejecutar, escribe tu predicción. Si no coincide con la salida, no borres el intento: anota qué supuesto era incorrecto y prueba una variación mínima. Al terminar, explica el concepto a otra persona usando un ejemplo propio.

### Punto de control inicial

Antes de avanzar, escribe dos ideas que ya conoces sobre este tema y una pregunta que todavía no sabes responder. Vuelve a ellas al final de la lección 10. Si cambias de respuesta, explica qué ejemplo o prueba hizo que cambiaras de opinión.

# Lección 1 · El modelo mental

## Idea central

Una comprensión construye una colección a partir de un iterable y puede incluir una condición. Un generador produce valores gradualmente: una expresión con paréntesis o una función con yield devuelve un iterador, no una lista ya calculada. Esto puede reducir memoria cuando se recorren muchos elementos, pero un generador normalmente se consume una vez. Elige la forma más clara para el tamaño y uso real de los datos.

Para razonar sobre comprensiones y generadores, separa cuatro preguntas: qué datos existen al empezar, qué regla se aplica, qué cambia durante el proceso y qué resultado debe observar quien usa el programa. Esta separación ayuda a no confundir la sintaxis con la finalidad. Una línea puede compilar y aun así resolver mal el problema si el dato representa otra cosa o falta un caso.

## En Python, ten presente este marco

Python asocia los tipos a los objetos y permite que una variable nombre valores de tipos distintos. int, float, str, bool y None aparecen desde el inicio; list, dict y set son mutables, mientras que tuple y str no se modifican en el sitio. La indentación delimita bloques. input() devuelve texto, por lo que convertir y validar son pasos diferentes.

## Comprueba tu comprensión

- ¿Qué parte de la explicación define la regla principal de comprensiones y generadores?
- ¿Qué dato puede cambiar y qué dato debería permanecer estable?
- ¿Qué observación concreta demostraría que tu programa cumple el objetivo?

# Lección 2 · Vocabulario y reglas

## Reglas de Python que afectan a este tema

list es una secuencia mutable; tuple es una secuencia inmutable; dict asocia claves hashables con valores; set representa elementos únicos. Una asignación puede crear otro nombre para la misma lista, no una copia. `in` comprueba pertenencia y su coste depende de la estructura elegida.

## Ideas específicas de comprensiones y generadores

Elige la estructura de datos según la operación que el programa necesite: una secuencia conserva posición, un conjunto representa pertenencia sin duplicados y un mapa asocia claves con valores. Distingue acceso por índice de acceso por clave y observa si la estructura es mutable. Si dos variables apuntan al mismo objeto mutable, cambiarlo desde una puede afectar a la otra.

## Glosario de trabajo

| Término | Significado en este recorrido |
| --- | --- |
| Secuencia | Estructura ordenada a la que se suele acceder por posición. |
| Clave | Dato usado para localizar un valor en un mapa o diccionario. |
| Mutabilidad | Posibilidad de cambiar un objeto después de crearlo. |
| Duplicado | Elemento repetido que una estructura puede conservar o descartar. |

No memorices la tabla como definiciones aisladas. Relaciona cada término con una línea del ejemplo o una decisión del ejercicio. Si una palabra aparece con otro significado en una API o en otra versión del lenguaje, anótalo junto a la fuente oficial antes de aplicarla al proyecto.

# Lección 3 · Lectura guiada del ejemplo

Primero identifica la entrada, luego sigue el valor por el programa. No supongas que una asignación transforma el dato: a veces solo enlaza un nombre con el mismo objeto. Observa dónde empieza cada bloque, qué condición activa una rama y qué instrucción produce la salida. Si el ejemplo tiene varias líneas, ejecútalo por partes y añade temporalmente observaciones seguras para ver los valores intermedios.

Código · python

```
def cuadrados_pares(numeros):
    for numero in numeros:
        if numero % 2 == 0:
            yield numero * numero

print(list(cuadrados_pares([1, 2, 4])))
```

## Salida esperada

```
[4, 16]
```

## Registro de la traza

| Momento | Qué anotar |
| --- | --- |
| Antes de ejecutar | Valores iniciales, sus tipos y qué operación esperas que ocurra. |
| Después de cada paso | Nombre afectado, valor nuevo y regla que produjo el cambio. |
| Al encontrar una decisión | Condición evaluada y camino que se ejecutó. |
| Al terminar | Salida real, diferencia respecto a tu predicción y causa. |
| Con una entrada distinta | Qué resultado cambia y qué parte del contrato permanece igual. |

## Traza en papel

- Antes de la primera instrucción: apunta los valores iniciales y sus tipos.
- Después de cada operación: anota solo los nombres que hayan cambiado y la razón.
- Al final: compara tu predicción con «[4, 16]» y explica la diferencia si la hay.

Cambia un dato de entrada y vuelve a hacer la traza. Una buena explicación debe anticipar el nuevo resultado, no limitarse a repetir la salida original.

# Lección 4 · Decidir cuándo usarlo

Elige la estructura de datos según la operación que el programa necesite: una secuencia conserva posición, un conjunto representa pertenencia sin duplicados y un mapa asocia claves con valores. Distingue acceso por índice de acceso por clave y observa si la estructura es mutable. Si dos variables apuntan al mismo objeto mutable, cambiarlo desde una puede afectar a la otra.

Caso aplicado: en un proyecto de DAM/DAW, una pantalla de matrículas necesita comprensiones y generadores. Describe qué datos de la matrícula intervienen, qué regla del tema usarías y qué resultado verá el usuario. Mantén el cálculo o decisión separado de la lectura de interfaz cuando sea posible; así podrás comprobar la regla sin abrir la pantalla.

### Compara antes de elegir

- ¿La solución más directa deja claro el propósito o requiere conocer demasiados detalles internos?
- ¿Qué coste aparece si crece el volumen de datos o se añade un nuevo caso?
- ¿Qué comprobación automatizada detectaría una modificación accidental?

### Enfoques relacionados

- Usar una expresión generadora en vez de una función con yield.
- Usar una comprensión de lista si se necesita guardar todos los resultados.

No existe una alternativa mejor en todos los contextos. Elige una y escribe su ventaja y su límite para el tamaño, la versión y los requisitos del ejercicio. Si dos enfoques cumplen exactamente el mismo contrato y ambos son legibles, pueden considerarse válidos.

# Lección 5 · Errores y diagnóstico

## Fallos que conviene reconocer

- Esperar que yield devuelva una lista completa de inmediato. Para localizarlo, reduce el caso a la menor entrada que lo reproduce, observa el valor implicado y verifica la regla antes de cambiar el código.
- Intentar recorrer dos veces un generador ya consumido. Para localizarlo, reduce el caso a la menor entrada que lo reproduce, observa el valor implicado y verifica la regla antes de cambiar el código.
- Escribir una comprensión tan anidada que resulte más difícil de leer que un bucle. Para localizarlo, reduce el caso a la menor entrada que lo reproduce, observa el valor implicado y verifica la regla antes de cambiar el código.

## Rutina de depuración

1. Describe el resultado esperado y el resultado real sin mezclar ambos. 2. Reproduce el fallo con una entrada fija. 3. Reduce el programa hasta dejar la primera operación que se comporta distinto. 4. Formula una hipótesis comprobable y cambia una sola cosa. 5. Añade una prueba que impida que el mismo fallo vuelva. Este orden también funciona cuando el error aparece en una API, una interfaz o un archivo.

## Preguntas de revisión

- ¿El error está en los datos, en la regla o en cómo se presenta el resultado?
- ¿Qué caso límite no estaba cubierto por el ejemplo inicial?
- ¿La solución corrige la causa o solo oculta el síntoma?

# Lección 6 · Práctica guiada

## Situación de DAM/DAW

Un flujo procesa identificadores y necesita producir, sin guardar toda la lista, los cuadrados de los valores positivos.

## Enunciado

Produce los cuadrados de los números positivos de una secuencia usando un generador.

## Convierte el enunciado en requisitos verificables

- Escribe qué entradas acepta la solución y qué debe devolver o mostrar.
- Enumera las reglas de negocio y separa las que son obligatorias de las decisiones de presentación.
- Antes de programar, completa la tabla de pruebas y predice cada resultado.

## Casos que debe superar

| Entrada | Resultado esperado |
|---|---|
| [] | [] |
| [-2, 0] | [] |
| [-1, 2, 3] | [4, 9] |

## Qué demuestra cada caso

Clasifica cada fila de la tabla como caso habitual, límite o variación importante. Si el enunciado no define qué hacer con una entrada fuera de contrato, no inventes un resultado silencioso: anota la ambigüedad y propone una regla antes de programar. En la práctica profesional, acordar esa regla evita que dos implementaciones parezcan correctas y se comporten de forma incompatible.

## Plan de solución

- Divide la tarea en pasos pequeños que puedan comprobarse por separado.
- Implementa primero el caso habitual y después añade los límites y errores previstos.
- Ejecuta cada caso tras un cambio; no esperes al final para descubrir qué parte falló.

# Lección 7 · Solución explicada

Una solución no se justifica solo porque la salida coincida en un ejemplo. Debe respetar el contrato para todas las entradas admitidas. Lee el código siguiente y vincula cada operación con una frase del enunciado. Si no puedes señalar esa relación, revisa el diseño antes de continuar.

Código · python

```python
def cuadrados_positivos(numeros):
    for numero in numeros:
        if numero > 0:
            yield numero ** 2
```

Criterio de aceptación: Se acepta una comprensión convertida a iterador si los valores se producen en el orden de entrada.

## Por qué se sigue este razonamiento

La función generadora recorre la entrada en su orden original, descarta los valores que no son mayores que cero y usa yield para emitir el cuadrado de cada positivo. Una entrada vacía o con -2 y 0 no emite nada; con -1, 2 y 3 emite 4 y 9. Para comparar con las listas esperadas hay que consumir el iterador, por ejemplo convirtiéndolo en lista.

## Comprobación de corrección

Comprueba cada caso de la tabla y explica qué parte de la solución lo cubre. Después revisa el caso mínimo, el máximo razonable y una entrada inválida si el contrato la permite. Comprueba también los tipos de entrada y salida: una coincidencia textual puede ocultar un valor mal tipado o una conversión con pérdida.

| Paso | Justificación que debes poder dar |
|---|---|
| Contrato | Qué valores acepta el ejercicio y qué resultado promete. |
| Regla | Qué operación transforma la entrada en la salida. |
| Pruebas | Qué caso comprueba lo normal y cuál descubre un límite. |

## Alternativas válidas

- Usar una expresión generadora en vez de una función con yield.
- Usar una comprensión de lista si se necesita guardar todos los resultados.

Al comparar soluciones, prioriza contrato correcto, legibilidad y pruebas. Una alternativa que cambia la representación interna puede ser válida; una que cambia silenciosamente el significado de la salida no lo es.

# Lección 8 · Laboratorio de aplicación

Retoma el ejercicio de comprensiones y generadores y amplíalo como una pequeña funcionalidad de una aplicación de gestión académica. Conserva el requisito original y añade una regla nueva que tenga sentido para los datos. Antes de modificar el código, escribe la regla en lenguaje natural, un ejemplo que la cumpla y otro que la incumpla.

### Variación para transferir lo aprendido

Consume el generador dos veces y observa que la segunda lectura no vuelve a producir valores; explica cuándo convendría materializarlo en una lista.

### Desafíos progresivos

- Nivel 1: añade un caso límite que no aparecía en la tabla y registra el resultado esperado.
- Nivel 2: cambia la forma de recibir o recorrer los datos sin cambiar el contrato externo.
- Nivel 3: separa la regla principal de su entrada/salida y prueba ambas partes de forma independiente.

### Criterios de entrega

- La solución conserva el resultado esperado del ejercicio original.
- El nuevo comportamiento queda cubierto por una prueba que falla antes del cambio.
- Los nombres permiten seguir el flujo sin comentarios que repitan la sintaxis.
- La entrada inválida o el error externo tiene una respuesta definida.

### Prueba de regresión

Guarda los casos originales antes de añadir la regla nueva. Ejecuta la batería completa después de cada cambio, no solo el nuevo caso. Si una prueba deja de pasar, decide si encontraste un defecto o si cambió el contrato y hay que actualizar la expectativa con una explicación. Termina con una nota breve: qué cambió, qué comprobaste y qué límite sigue abierto.

Anota qué parte se puede reutilizar en otro módulo y qué parte depende del contexto de la aplicación. Esa distinción ayuda a decidir si merece la pena extraer una función o componente.

# Lección 9 · Repaso activo

Cierra el material y responde sin copiar frases. Si dudas, vuelve a la página indicada, comprueba la fuente primaria y reescribe la respuesta con tus palabras. La recuperación activa ayuda a detectar qué parte aún depende de memorizar un ejemplo concreto.

## Preguntas

- ¿Qué problema aborda comprensiones y generadores y qué dato necesita?
- ¿Qué diferencia hay entre una solución que funciona en el ejemplo y una que cumple el contrato?
- ¿Qué error frecuente te parece más probable y cómo lo reproducirías?
- ¿Qué alternativa elegirías en un proyecto pequeño y por qué?

## Respuestas de comprobación

- La idea clave se resume en esta explicación: Una comprensión construye una colección a partir de un iterable y puede incluir una condición. Un generador produce valores gradualmente: una expresión con paréntesis o una función con yield devuelve un iterador, no una lista ya calculada. Esto puede reducir memoria cuando se recorren muchos elementos, pero un generador normalmente se consume una vez. Elige la forma más clara para el tamaño y uso real de los datos.
- Una fuente inicial de fallo descrita en el tema es: Esperar que yield devuelva una lista completa de inmediato.
- Una alternativa es válida si mantiene las entradas, salidas y reglas acordadas y supera las mismas pruebas.

Si tu respuesta no menciona un caso o una condición concreta, añade un ejemplo propio y vuelve a comprobarlo.

# Lección 10 · Síntesis y fuentes

## Resumen del tema

Has estudiado comprensiones y generadores desde el modelo conceptual hasta una práctica comprobable. La secuencia de trabajo es: identificar entradas y contrato, elegir la operación adecuada, seguir el flujo, comprobar casos límite y conservar una prueba para la siguiente modificación. Aplica las reglas de Python sin confundir una característica del lenguaje con una API concreta del entorno.

## Lista de control antes de avanzar

- Puedo explicar el concepto con un ejemplo propio y predecir su salida.
- Puedo identificar al menos un caso límite y un error habitual.
- Puedo comparar dos enfoques y justificar mi elección.
- Puedo verificar una solución sin depender de mirar el resultado una sola vez.

## Fuentes oficiales para ampliar

Python Software Foundation — Data Structures · Python Software Foundation — Generators

Consulta la sección de la fuente que corresponda a la versión de tu proyecto. Este cuaderno resume conceptos para estudiar; cuando haya una diferencia de detalle, prevalece la documentación oficial del lenguaje o biblioteca.